RAG Learning

4.1 Fundamentals of Programming

Computer Science — AQA A-Level (7517) — Paper 1: On-screen Programming

Topic 4.1 underpins **everything** in Paper 1 and is also the basis for the on-screen coding tasks. You must be able to *write, trace and explain* code using AQA pseudocode or your chosen high-level language (Python, VB.NET, C#, Java, Pascal). Expect questions that ask you to identify data types, predict output from a trace, explain scope, compare iteration with recursion, and justify design choices in object-oriented code.

4.1.1 Data Types

Every variable in a program is of some **data type**. The type determines what values can be stored, what operations are legal on the value, and how many bits it occupies in memory.

> A **data type** is a classification that specifies the set of permitted values a variable may hold, the operations that may be performed on those values, and the way they are represented in memory.

Primitive (Built-in) Data Types

Type	Description	Typical size	Example
`integer`	A whole number, positive or negative, with no fractional part. Usually stored in two's complement.	32 or 64 bits	`age ← 17`
`real` / `float`	A number with a fractional part, stored in IEEE 754 floating-point form (sign, exponent, mantissa).	32 bits (single) or 64 bits (double)	`pi ← 3.14159`

`Boolean`	A logical value: either `True` or `False`. Only 1 bit is required logically but a whole byte is usually allocated.	1 byte	`isValid ← True`
`character`	A single symbol from a character set (ASCII = 7 bits, extended ASCII = 8 bits, Unicode UTF-8 = 1–4 bytes).	1–4 bytes	`grade ← 'A'`
`string`	An ordered sequence of characters. Usually stored either as a length-prefixed array or a null-terminated array.	variable	`name ← "Ada"`
`date/time`	A composite of year, month, day (and optionally time). Often stored as a count of seconds since an epoch.	8 bytes typical	`dob ← #01/05/2008#`

How Each Type is Represented in Binary

- **Integer** — stored using **two's complement**. In an 8-bit signed integer the range is −128 to +127. The most significant bit is the sign bit; negative numbers have a '1' MSB.
- **Real** — stored in **floating-point** form: a sign bit, an exponent (normally biased), and a mantissa (normalised so the leading bit is implicit). Precision is finite, so many decimal fractions (e.g. 0.1) cannot be stored exactly.
- **Boolean** — a single bit (0 = False, 1 = True) padded to a byte for addressability.
- **Character** — mapped to an integer code using a character set such as **ASCII** (0–127) or **Unicode**. For example `'A'` = 65 = `01000001`.
- **String** — a contiguous array of character codes, usually preceded by a length word or terminated by a zero byte.

> **Exam Tip:** AQA frequently asks why `0.1 + 0.2` does not equal exactly `0.3`. The answer is that `0.1` and `0.2` are recurring fractions in binary, so they are stored with a small rounding error. Always mention *finite mantissa* for full marks.

Type Coercion vs Casting

> **Type coercion** (or *implicit conversion*) is when the compiler / interpreter *automatically* converts one data type to another, for instance promoting an integer to a real in the expression `3 + 2.5`. **Casting** (or *explicit conversion*) is when the programmer forces a conversion using syntax such as `int(x)` or `(float) n`.

Coercion is convenient but can hide bugs (e.g. `"5" + 3` concatenates in some languages and errors in others). Explicit casting makes intent clear and is safer in production code.

User-Defined Data Types

Primitives can be combined or restricted to create new named types.

- **Enumerated type** — an ordered list of named values. The compiler substitutes integer codes behind the scenes. Useful for fixing a closed set of possibilities.

```
TYPE Day = (Mon, Tue, Wed, Thu, Fri, Sat, Sun)
today : Day
today ← Wed
```

- **Record / composite type** — groups several named fields, each with its own type. Equivalent to a `struct` in C or a simple class without methods.

```
TYPE Student = RECORD
    name    : String
    year    : Integer
    average : Real
ENDRECORD

s : Student
s.name ← "Ada"
s.year ← 13
```

- **Subrange / set type** — a primitive constrained to a range, e.g. `mark : 0..100`.
- **Pointer / reference types** — hold the memory address of another value.

> **Exam Tip:** If a question asks you to design a data structure (e.g. for a library book or a bank account), use a **record** with clearly named, type-annotated fields. Examiners love to see `:`

> followed by a type.

4.1.2 Programming Concepts

Variable and Constant Declaration

A **variable** is a named memory location whose value may change during execution. A **constant** is a named value that is fixed at declaration and cannot subsequently be altered.

```
CONSTANT VAT_RATE : Real = 0.20
DECLARE price    : Real
price ← 49.99
total ← price * (1 + VAT_RATE)
```

The Three Basic Constructs

All procedural programs can be built from just three building blocks — the **structured programming theorem** of Böhm and Jacopini (1966).

- **Sequence** — statements executed one after another, top-to-bottom.
- **Selection** — choosing between alternative paths using `IF / ELSE IF / ELSE` or `CASE SELECT`.
- **Iteration** — repeating a block of code.
  - **Definite** — fixed number of repetitions, e.g. `FOR i ← 1 TO 10`.
  - **Indefinite** — repetition controlled by a Boolean condition, e.g. `WHILE / REPEAT UNTIL`.

```
// Selection
IF mark ≥ 70 THEN
    grade ← "A"
ELSE IF mark ≥ 60 THEN
    grade ← "B"
ELSE
    grade ← "C"
ENDIF

// Definite iteration
FOR i ← 1 TO 5
```

```
    OUTPUT i * i
ENDFOR

// Indefinite iteration
WHILE NOT validInput DO
    input ← USERINPUT
    validInput ← (input ≥ 0)
ENDWHILE
```

Nested Constructs

Any of the three constructs may be nested inside any other. For example, a `WHILE` loop whose body contains an `IF` statement whose branch contains a `FOR` loop. Indentation is essential for readability — examiners deduct marks for unreadable nested code.

Meaningful Identifier Names

- Self-documenting: `numberOfStudents` is clearer than `n`.
- Case convention: most languages use *camelCase* for variables, *PascalCase* for types, *SCREAMING_SNAKE_CASE* for constants.
- Avoid reserved words, accents, or names that differ by one character only.
- Scope-appropriate: short names (`i` , `j`) are fine as loop counters in a tight scope; long descriptive names belong to longer-lived variables.

4.1.3 Arithmetic Operations

Beyond the usual `+ - * /`, AQA pseudocode defines:

Operator	Meaning	Example	Result
`DIV`	Integer division — discards remainder	`17 DIV 5`	`3`
`MOD`	Modulo — remainder after integer division	`17 MOD 5`	`2`
`/` (real)	Real division	`17 / 5`	`3.4`

`^` or `**`	Exponentiation	`2 ^ 10`	`1024`
`ROUND(x, d)`	Round x to d decimal places (banker's rounding in some languages)	`ROUND(3.14159, 2)`	`3.14`
`TRUNCATE(x)`	Drop the fractional part (always towards zero)	`TRUNCATE(3.9)`	`3`

Classic use of MOD — testing divisibility: `IF n MOD 2 = 0 THEN "even"`.
Classic use of DIV — extracting digits: `tens = (n DIV 10) MOD 10`.

> **Exam Tip:** Be careful with *negative* DIV and MOD: languages differ. Python defines `-7 MOD 3` as `2` (result sign matches divisor), whereas C defines it as `-1` (result sign matches dividend). In AQA questions, assume the Python convention unless told otherwise.

4.1.4 Relational Operations

Relational operators compare two values and return a Boolean.

AQA symbol	Most-language syntax	Meaning
`=`	`==`	Equal to
`≠`	`!=` / `<>`	Not equal to
`<`	`<`	Less than
`>`	`>`	Greater than
`≤`	`<=`	Less than or equal to
`≥`	`>=`	Greater than or equal to

Strings are typically compared *lexicographically*, comparing character codes position by position. `"apple" < "banana"` is `True` because `'a' < 'b'`.

4.1.5 Boolean Operations

Boolean operators combine or invert truth values.

Truth Tables

A	B	A AND B	A OR B	A XOR B	NOT A
F	F	F	F	F	T
F	T	F	T	T	T
T	F	F	T	T	F
T	T	T	T	F	F

Short-circuit evaluation — most languages evaluate `A AND B` left-to-right and stop as soon as the answer is determined. If `A` is false, `B` is never evaluated. This is why `IF list ≠ NULL AND list.length > 0` is safe: the right-hand side is skipped when the pointer is null.

> **XOR** (exclusive OR) is true when *exactly one* of the inputs is true. It is equivalent to `(A OR B) AND NOT(A AND B)`, and also to `A ≠ B` for Boolean inputs. XOR is its own inverse: `(x XOR k) XOR k = x`, which underlies simple stream ciphers.

De Morgan's laws (useful for simplifying complex conditions):

- `NOT (A AND B) = (NOT A) OR (NOT B)`
- `NOT (A OR B) = (NOT A) AND (NOT B)`

4.1.6 Constants and Variables

Use a **constant** whenever a value:

- has a known meaning at compile time (e.g. `PI`, `VAT_RATE`, `MAX_PLAYERS`);

- must be the *same* everywhere it is used;
- must not be altered even accidentally.

Advantage of constants	Why it matters
Readability	`total = price * VAT_RATE` is clearer than `total = price * 0.20` — the reader doesn't need to guess what `0.20` means.
Single source of truth	If the VAT rate changes to 0.23, one edit changes every computation. Scattered magic numbers risk being updated inconsistently.
Prevents accidental modification	The compiler refuses any assignment to the constant after declaration, eliminating a whole class of bugs.
Enables optimisation	The compiler can inline constants and perform constant-folding, producing faster code.

Exam Tip: "Give two advantages of using a constant" is a very common 2-mark question. Do *not* answer "it saves memory" — constants do not save memory. Use *readability* and *single point of change*.

4.1.7 Structured Programming

Structured programming is a programming paradigm that encourages programs to be built from the three control constructs (sequence, selection, iteration), broken down (*decomposed*) into self-contained subroutines, avoiding unrestricted jumps such as `GOTO`.

Its benefits include:

- **Easier to read, debug and maintain** — every subroutine has one entry and one exit point.
- **Reusable** — a well-scoped subroutine can be called from many places or reused in other programs.

- **Testable in isolation** — each module can be unit-tested independently.
- **Parallel teamwork** — different programmers can work on different modules concurrently once the interfaces are agreed.

Top-down (stepwise) design starts with a single high-level statement of the problem, then decomposes it into smaller sub-problems, repeating until each sub-problem is trivially codeable. The resulting hierarchy is often drawn as a *structure chart*.

4.1.8 Subroutines (Procedures and Functions)

> A **subroutine** is a named, self-contained block of code that performs a specific task and may be called (invoked) from elsewhere in the program. A **procedure** carries out an action but returns no value; a **function** returns a single value and is normally free of side effects.

```
PROCEDURE greet(name : String)
    OUTPUT "Hello, " + name
ENDPROCEDURE

FUNCTION square(x : Integer) RETURNS Integer
    RETURN x * x
ENDFUNCTION

greet("Ada")          // procedure call — no value expected
y ← square(7)        // function call — y becomes 49
```

Benefits of subroutines echo the benefits of structured programming: **abstraction** (hide implementation), **reuse** (write once, call many), **testability**, and **maintainability**.

4.1.9 Parameters

A **parameter** is a variable listed in the subroutine's definition that receives a value when the subroutine is called.

- **Formal parameter** — the name used in the subroutine's header (e.g. `x` in `FUNCTION square(x)`).
- **Actual parameter / argument** — the value passed in by the caller (e.g. `7` in `square(7)`).

Passing Mechanisms

Aspect	Pass by Value	Pass by Reference
What is copied	The *value* of the argument is copied into a new local parameter.	The *address* of the argument is passed; the parameter becomes an alias for the original.
Effect on caller	The original variable is unchanged, even if the parameter is reassigned inside the subroutine.	Assignments to the parameter inside the subroutine are visible to the caller.
Memory overhead	Copies the whole value — potentially expensive for large records.	Only an address is copied — cheap regardless of data size.
Side effects	None on the caller's data.	Possible — parameter changes *leak* back to the caller.
Typical use	Simple inputs: integers, small structs, values that the subroutine should not alter.	Large objects, arrays, or when the subroutine must modify its argument.

```
PROCEDURE addOne(BYVAL n : Integer)
    n ← n + 1
ENDPROCEDURE

PROCEDURE addOneRef(BYREF n : Integer)
    n ← n + 1
ENDPROCEDURE

x ← 5
addOne(x)       // x is still 5
addOneRef(x)    // x is now 6
```

> **Exam Tip:** When asked to *compare* the two mechanisms, structure your answer as a table or as two bullet lists. Use the word **alias** when describing pass-by-reference and the word **copy** when describing pass-by-value — these are the exact words on AQA mark schemes.

Side Effects

> A **side effect** is any change a subroutine makes to state outside its own local variables — for example modifying a global variable, writing to a file, printing to the screen, or (via pass-by-reference) altering its caller's arguments.

A *pure function* has no side effects and always returns the same output for the same input. Pure functions are easier to test, parallelise, and reason about. Procedures that perform I/O or mutate reference parameters are, by definition, not pure.

4.1.10 Returning Values

A function uses `RETURN value` to deliver a single result back to the caller; at that point the subroutine ends and control resumes at the caller site.

```
FUNCTION absolute(n : Integer) RETURNS Integer
    IF n < 0 THEN
        RETURN -n
    ELSE
        RETURN n
    ENDIF
ENDFUNCTION

d ← absolute(-7)   // d is 7
```

Languages differ in how they handle multiple return values:

- Python — returns a tuple: `return a, b`.
- C/C++ — returns one value; uses pointer / reference parameters for "out" values.
- Java / C# — returns one value; use a record, object, or `out` parameters for more.

4.1.11 Local and Global Variables

The **scope** of a variable is the region of a program in which that identifier is visible. A **local variable** is declared inside a subroutine and exists only for the lifetime of that call. A **global variable** is declared at module level and is visible to every subroutine.

```
total ← 0             // global

PROCEDURE addItem(price : Real)
    tax ← price * 0.2   // local
    total ← total + price + tax
ENDPROCEDURE
```

Why Globals are Discouraged

- **Hidden dependencies** — any subroutine could silently depend on a global, making reasoning difficult.
- **Tight coupling** — components can no longer be reused in isolation.
- **Name clashes** — a global can be shadowed accidentally by a local of the same name.
- **Thread-unsafe** — concurrent access to a shared global requires explicit locking.
- **Harder to test** — a unit test must set up (and tear down) global state.

Shadowing

If a local variable is declared with the *same name* as an outer variable, the local *shadows* the outer for the duration of the block. References to that name bind to the nearest enclosing declaration.

```
x ← 1            // global x

PROCEDURE demo()
    x ← 99       // new local x (in most languages); shadows global
    OUTPUT x      // prints 99
ENDPROCEDURE
```

© RAG Learning · Not for redistribution

```
demo()
OUTPUT x          // prints 1
```

> **Exam Tip:** AQA routinely tests trace questions that hinge on scope. Always ask: "is this identifier declared inside the current subroutine?" If yes, it is local; if not, walk outwards to the next enclosing scope. Globals are the final fallback.

4.1.12 The Role of Stack Frames in Subroutine Calls

> A **stack frame** (or *activation record*) is a block of memory pushed onto the call stack for every subroutine invocation. It contains the parameters, local variables, saved registers, and the **return address** to which control should jump when the subroutine completes.

Each call pushes a new frame; each `RETURN` pops the topmost frame. This explains both **how recursion works** and **why it can overflow**.

A typical stack frame layout (growing downwards) contains:

```
|   caller's locals ...   |
|-------------------------|
|  return address (RA)    |   ← where to jump back to
|-------------------------|
|  saved frame pointer    |
|-------------------------|
|  parameters             |   ← value or reference data
|-------------------------|
|  local variables        |
|-------------------------|
|  (top of stack)         |
```

Walkthrough — calling `square(7)` from main:

1. Main evaluates the argument `7` and pushes it.
2. The CPU executes a CALL instruction which pushes the *return address* (the next instruction after the call).

3. `square` allocates space for its local variables by adjusting the stack pointer.
4. `square` computes `7 * 7 = 49` and places the result in the return register.
5. `RETURN` pops the local variables, then the return address, and jumps to it.
6. Main resumes with `49` available for assignment.

4.1.13 Recursion

> A subroutine is **recursive** if it calls itself (directly or indirectly). Every recursive subroutine must have at least one **base case** that stops the recursion and at least one **recursive case** that calls the subroutine with a simpler / smaller argument, making *progress* towards the base case.

Worked Example: Factorial

```
FUNCTION factorial(n : Integer) RETURNS Integer
    IF n = 0 THEN                   // base case
        RETURN 1
    ELSE                            // recursive case
        RETURN n * factorial(n - 1)
    ENDIF
ENDFUNCTION
```

Trace for `factorial(4)`:

Call depth	Call	Waiting on	Returns
0	factorial(4)	4 × factorial(3)	24
1	factorial(3)	3 × factorial(2)	6
2	factorial(2)	2 × factorial(1)	2
3	factorial(1)	1 × factorial(0)	1
4	factorial(0)	— base case —	1

© RAG Learning · Not for redistribution

At the deepest point, five stack frames are live simultaneously. As the base case returns `1`, each outer frame multiplies and returns, collapsing the stack.

Worked Example: Fibonacci Call Tree

```
FUNCTION fib(n : Integer) RETURNS Integer
    IF n < 2 THEN
        RETURN n
    ELSE
        RETURN fib(n - 1) + fib(n - 2)
    ENDIF
ENDFUNCTION
```

`fib(4)` expands into the following (exponentially branching) call tree, demonstrating why naive recursion can be *catastrophically* slow:

```
                fib(4)
              /        \
          fib(3)        fib(2)
         /      \       /     \
      fib(2)  fib(1)  fib(1)  fib(0)
      /   \
   fib(1) fib(0)
```

Note that `fib(2)` is recomputed twice and `fib(1)` three times. Running time is $O(\phi^n) \approx O(1.618^n)$, which is why this definition should be memoised or rewritten iteratively in practice.

Recursion vs Iteration

Aspect	Recursion	Iteration
Code clarity	Often more natural for inherently recursive problems (trees, divide-and-conquer, fractals).	Often clearer for linear, step-by-step processes (counting loops).
Memory	Every call uses a stack frame; deep recursion may trigger a **stack overflow**.	Fixed memory — only the loop variables occupy space.

Speed	Function-call overhead (pushing / popping frames) costs extra time.	No call overhead — typically faster.
Termination	Base case needed; easy to forget, leading to infinite recursion.	Loop condition needed; easier to get right.
Data structures	Maps naturally onto recursive structures (trees, linked lists).	Requires an *explicit* stack to simulate traversal.

Exam Tip: When asked "give one advantage of recursion over iteration", a safe answer is "*recursive code is often shorter and more readable for problems defined recursively, such as tree traversal*". For the disadvantage, mention *stack overflow* for large inputs.

Tree Traversal Example

```
PROCEDURE inorder(node)
    IF node ≠ NULL THEN
        inorder(node.left)
        OUTPUT node.value
        inorder(node.right)
    ENDIF
ENDPROCEDURE
```

This three-line recursive solution mirrors the recursive definition of a binary tree. An iterative equivalent requires an explicit stack and roughly twice the code.

4.1.14 Object-Oriented Programming

Object-oriented programming (OOP) is a paradigm in which a program is organised as a set of *objects* — self-contained units that combine **state** (attributes) and **behaviour** (methods) — that interact by sending messages (calling methods) to one another.

Core Terminology

Term	Definition
Class	A blueprint that defines the attributes and methods common to a category of objects.
Object / instance	A concrete value created from a class, with its own copy of the attribute values.
Attribute (field)	A named piece of state held by an object.
Method	A subroutine defined inside a class that operates on objects of that class.
Constructor	A special method that initialises a newly created object.
Encapsulation	Bundling state and behaviour together and hiding internal state behind a public interface (getters / setters). Attributes are usually `private`; methods are `public`.
Inheritance	Defining a new class (the *subclass / child*) by extending an existing one (the *superclass / parent*), automatically inheriting its attributes and methods.
Polymorphism	Calling the same method name on different classes and getting class-appropriate behaviour. Achieved by *overriding* (subclass redefines a method) or *overloading* (same name, different parameter list).
Association	A general "uses-a" relationship between independent classes.
Aggregation	A "has-a" relationship where the part can exist without the whole (e.g. a *Team* has *Players*; players still exist if the team is deleted).
Composition	A stronger "owns-a" relationship where the part cannot exist without the whole (e.g. a *Book* composed of *Pages*; delete the book and the pages go too).

Encapsulation Example

```
CLASS BankAccount
    PRIVATE balance : Real
```

```
    PUBLIC PROCEDURE new(opening : Real)
        balance ← opening
    ENDPROCEDURE

    PUBLIC PROCEDURE deposit(amount : Real)
        IF amount > 0 THEN
            balance ← balance + amount
        ENDIF
    ENDPROCEDURE

    PUBLIC FUNCTION getBalance() RETURNS Real
        RETURN balance
    ENDFUNCTION
ENDCLASS
```

External code can only modify `balance` via `deposit`, which validates the amount. The invariant "balance is never illegally changed" is enforced by the class itself — a huge advantage over plain records.

Inheritance and Polymorphism Example

```
CLASS Shape
    PROTECTED name : String

    PUBLIC PROCEDURE new(n : String)
        name ← n
    ENDPROCEDURE

    PUBLIC FUNCTION area() RETURNS Real
        RETURN 0
    ENDFUNCTION

    PUBLIC PROCEDURE describe()
        OUTPUT name + " has area " + area()
    ENDPROCEDURE
ENDCLASS

CLASS Circle INHERITS Shape
    PRIVATE radius : Real

    PUBLIC PROCEDURE new(r : Real)
```

```
        SUPER.new("Circle")
        radius ← r
    ENDPROCEDURE

    PUBLIC FUNCTION area() RETURNS Real     // override
        RETURN 3.14159 * radius * radius
    ENDFUNCTION
ENDCLASS

CLASS Square INHERITS Shape
    PRIVATE side : Real

    PUBLIC PROCEDURE new(s : Real)
        SUPER.new("Square")
        side ← s
    ENDPROCEDURE

    PUBLIC FUNCTION area() RETURNS Real     // override
        RETURN side * side
    ENDFUNCTION
ENDCLASS

// Polymorphic usage
shapes : ARRAY OF Shape
shapes[1] ← NEW Circle(3)
shapes[2] ← NEW Square(4)

FOR EACH s IN shapes
    s.describe()          // calls the correct area() automatically
ENDFOR
```

The call `s.describe()` is identical for every element, yet it prints the right area for each shape. This is **dynamic dispatch** in action.

UML Class Diagram (simplified notation)

```
+---------------------+
|        Shape        |
+---------------------+
| # name : String     |    # = protected
+---------------------+
| + area() : Real     |    + = public
```

```
| + describe()        |
+---------------------+
          ⇑
   -----------------
   |               |
+---------+   +-----------+
| Circle  |   |  Square   |
+---------+   +-----------+
| - radius|   | - side    |    - = private
+---------+   +-----------+
| + area()|   | + area()  |
+---------+   +-----------+
```

Overriding vs Overloading

	Overriding	Overloading
Where it happens	Across a class hierarchy — subclass re-implements a superclass method.	Within a single class — two methods share a name but differ in parameter list.
Signature	Must match the superclass method's signature exactly.	Must differ in number or type of parameters.
Resolved	At runtime (dynamic dispatch).	At compile time (static dispatch).
Purpose	Customise inherited behaviour.	Provide multiple ways to invoke a single conceptual operation.

OOP Design Principles

- **Encapsulation** — hide internal state; expose a minimal, well-defined interface.
- **DRY (Don't Repeat Yourself)** — every piece of knowledge should have one authoritative representation in the code. Duplicated code is a bug magnet.
- **SRP (Single Responsibility Principle)** — a class should have one, and only one, reason to change.
- **Liskov Substitution Principle** — objects of a subclass should be usable wherever their superclass is expected, without surprising the caller.

© RAG Learning · Not for redistribution

- **Open / Closed Principle** — classes should be open for extension (via inheritance or composition) but closed for modification once in use.
- **Favour composition over inheritance** — deep inheritance hierarchies become brittle; composing smaller objects is often more flexible.

> **Exam Tip:** For "explain encapsulation" questions, always give two parts: (1) bundling data and methods together in one class, and (2) restricting direct access to the data via access modifiers. Top-mark answers also cite a benefit — easier maintenance or enforcement of invariants.

4.1.15 Exception Handling

> An **exception** is an abnormal event that disrupts the normal flow of a program, for example a division by zero, a file not found, or invalid user input. **Exception handling** is the language feature that lets a program detect such an event, respond to it cleanly, and (optionally) recover.

The try / except / finally Construct

```
TRY
    age ← INT(USERINPUT)
    IF age < 0 THEN
        RAISE ValueError("age cannot be negative")
    ENDIF
    OUTPUT "In 10 years you'll be ", age + 10
EXCEPT ValueError AS e
    OUTPUT "Bad value: ", e
EXCEPT DivideByZeroError
    OUTPUT "Cannot divide by zero"
FINALLY
    OUTPUT "Thank you for using the age calculator."
ENDTRY
```

- **TRY** — the block of code that might raise an exception.
- **EXCEPT** (or `CATCH`) — a handler for a specific exception type. Multiple EXCEPT blocks may follow a single TRY, in order of specificity.

- **FINALLY** — an optional block that always executes, whether or not an exception occurred. Used for cleanup (closing files, releasing locks) that must happen in every case.
- **RAISE** (or `THROW`) — actively signals an exception, typically with an error message.

Common Exception Types

Exception	Typical cause
`ValueError` / `FormatException`	Converting "abc" to Integer.
`TypeError`	Adding a Boolean to a String.
`IndexOutOfRange`	Accessing `arr[10]` when the array has only 5 elements.
`KeyError` / `KeyNotFound`	Looking up an absent dictionary key.
`DivideByZeroError`	Integer division or MOD where the divisor is zero.
`FileNotFoundError`	Opening a file path that does not exist.
`NullReferenceException`	Calling a method on a null / None object.
`StackOverflowError`	Recursion depth exceeds the call stack's limit.

Exceptions vs Return Codes

Approach	Strengths	Weaknesses
Exceptions	Separate normal logic from error handling; propagate automatically up the call stack; force the programmer to acknowledge failures (in checked-exception languages).	Throwing / catching is slower than a conditional test; abused exceptions create "goto-like" flow that is hard to follow; finally / cleanup is easy to forget.
Return codes	Very fast (just an integer check); explicit — the reader sees every failure path.	Clutter normal logic with repeated `if result = ERROR` checks; a missed check silently continues with bad data.

© RAG Learning · Not for redistribution

Rule of thumb: use exceptions for *genuinely exceptional* conditions (file missing, network timeout, corrupt input) and use normal control flow for *expected* branches (e.g. "is the user a member?").

> **Exam Tip:** If a 6-mark question asks you to "explain how and why exception handling is used", award yourself marks for: (1) definition of an exception; (2) try / catch / finally mechanism; (3) separation of concerns (normal logic vs error handling); (4) an example; (5) a named common exception; (6) a comparison with old-fashioned return-code error handling.

Summary Checklist

- Name and describe all six primitive data types and at least two user-defined types.
- Distinguish type coercion from casting with a concrete example.
- Evaluate expressions using DIV, MOD, relational and Boolean operators.
- Construct truth tables for AND, OR, NOT, XOR.
- Justify the use of constants.
- Write and trace subroutines with formal and actual parameters, using both pass-by-value and pass-by-reference.
- Explain local vs global scope and shadowing.
- Describe a stack frame's contents and how recursion uses the stack.
- Write, trace and critique a recursive algorithm; compare it with an iterative equivalent.
- Use class, object, attribute, method, constructor, encapsulation, inheritance, polymorphism, association, aggregation and composition correctly in a written answer.
- Use try / except / finally to handle a named exception and explain why it is preferable to return codes in that situation.

© RAG Learning · Not for redistribution